Sematics-Preserving Data Layout Transformations for Improved Vectorisation

Artjoms Šinkarovs
Heriot-Watt University
a.sinkarovs@macs.hw.ac.uk

Sven-Bodo Scholz
Heriot-Watt University
s.scholz@hw.ac.uk

Abstract

Data-Layouts that are favourable from an algorithmic perspective often are less suitable for vectorisation, i.e., for an effective use of modern processor's vector instructions. This paper presents work on a compiler driven approach towards automatically transforming data layouts into a form that is suitable for vectorisation. In particular, we present a program transformation for a first-order functional array programming language that systematically modifies they layouts of all data structures. At the same time, the transformation also adjusts the code that operates on these structures so that the overall computation remains unchanged. We define a correctness criterion for layout modifying program transformations and we show that our transformation abides to this criterion.

1. Introduction

Most programming languages directly relate type definitions and type declarations to one particular layout of the data in memory. For example, FORTRAN maps arrays in a column major order into memory, whereas C uses a row major scheme. The fields of records or objects are typically adjacent in memory and algebraic data types such as those in many functional languages typically are mapped to pointer connected graphs in memory.

In languages that support an explicit notion of memory and pointers there is very little that can be done about this tight coupling between types and their corresponding mapping into memory. In contrast, language that completely abstract away from the notion of memory, such as purely functional languages, allow for almost arbitrary mappings of data into memory. While this opportunity may be less relevant in the context of algebraic data types, it definitely can have a huge impact on the performance of programs that operate on arrays.

It is well known from the optimisation of compute intensive applications in FORTRAN or C, that a reorganisation of data accesses can vastly improve the overall runtime [1, 7, 11] Improvements typically do not only stem from better cache locality but also from improved chances of the utilisation of vector instructions, i.e., auto-vectorisation. However, data dependencies in programs and a fixed data-layout often constrain what can be achieved. The high-performance computing literature provides many cases where manual re-writes for enforcing different memory layouts is crucial for achieving a sufficient level of performance [6, 8].

Here, a purely functional setting offers a unique opportunity. The implicitness of the memory management allows the compiler to adjust a programmer-specified layout in a way that improves locality and that enables better vectorisation opportunities. In [12] we have demonstrated for a small functional core language how a type system can be used to ensure consistent layout adjustments, we sketched an inference algorithm and we demonstrated the potential of the approach by looking at the N-body problem and an implementation in SAC. In this paper, we present a formal layout-transformation scheme that takes a program annotated with layout-types and transforms the program on the source level so that a straight-forward mapping into a pre-defined memory mapping achieves the desired layout transformation. This approach allows layout transformations to be implemented as a high-level transformation without requiring any adjustments on the low-level code generation. Besides the type-driven code transformation scheme we also provide a correctness proof of our transformation. It is based on a correctness notion that captures the essence of data-layout independence by looking at equality as being factored by index-space transformations of function arguments and function results.

The rest of the paper is organised as follows: the next section formalises the programming language we use to describe our transformation, briefly introcuces the layout type system and formailises correctness criteria. Section 3 introcuces the transformation itself providing informal explanation across the lines. Section 4 gives a formal proof of the correctness of our transformation according to the criteria formulated earlier. Then follows a brief example of application of the described technique using matrix multiplication as a case study. Section 6 discusses related work and we conclude with Section 7.

2. Setting

For our formalisms we use a first order functional core language. It is a stripped-down version of SAC [2] very similar to the core language introduced in [3]. In essence, the language constitutes an applied λ-calculus with a C flavour when it comes to the syntax for function definitions and function applications. Furthermore, it contains the core constructs for defining arrays and data-parallel operations on them.

Fig. 1 shows the syntax of SAC-λ. The overall goal expression is preceded by a set of mutually recursive function definitions. Constants can either be scalars or vectors of expressions, each of which should evaluate to arrays of identical shape. Function applications are written in a C style for both, user defined functions and for built-in functions (*P*rf).

The built-in functions are mainly responsible for arithmetic operations and for element-selections. For convenience we use $e[iv]$ interchangeably with sel (iv, e). Central to the data-parallelism

[Copyright notice will appear here once 'preprint' option is removed.]

$$
\begin{array}{lcl}
Program & \Rightarrow & \textbf{letrec} \left[FunDef \right]^* \textbf{ in } E \\
FunDef & \Rightarrow & FunId \, (\left[Id \left[\, , Id \right]^* \right]) \, = E \\
E & \Rightarrow & Const \mid Id \mid FunId \, (\left[E \left[\, , E \right]^* \right]) \\
 & \mid & Prf \, (\left[E \left[\, , E \right]^* \right]) \\
 & \mid & \textbf{if } E \textbf{ then } E \textbf{ else } E \\
 & \mid & \textbf{let } Id = E \textbf{ in } E \\
 & \mid & \textbf{map } Id < E \; E \\
 & \mid & \textbf{reduce } Id < E \; (\; FunId \;) \; E \\
Const & \Rightarrow & c \mid \; [\left[E \left[\, , E \right]^* \right]] \\
Prf & \Rightarrow & \textbf{sel} \mid + \mid - \mid ...
\end{array}
$$

Figure 1. The syntax of SAC-λ

support in SAC-λ are the language constructs `map` and `reduce`, which are array versions of their well-known counterparts on lists. While `map` defines all elements of an array `reduce` defines a fold operation over an index range. Index ranges, for both these operations, are specified directly after the keywords `map` and `reduce`], respectively. They are denoted by an identifier which will range over the index space, followed by the symbol `<`, followed by the upper bound of the index range.

2.1 Semantics of SAC-λ

Every value in SAC-λ, conceptually, is a multi-dimensional array, which is being represented as a pair: $\langle s, d \rangle$, where s is a shape and d is data. Both shape and data are vectors, which are denoted in square brackets, e.g. $[1, 2, 3]$. Scalar constants are represented as arrays with empty shapes. Further down we are going to define each term formally using big-step semantics. Scalar constants are defined as:

$$\text{SCALAR}: \frac{}{n \Downarrow \langle \, [], [n] \, \rangle}$$

For non-scalar constants the following semantics holds:

$$\text{VECT}: \frac{\forall i \in \{1, \ldots, n\} : e_i \Downarrow \langle \, [s_1, \ldots, s_m], [d_1^i, \ldots, d_p^i] \, \rangle}{\begin{array}{c} v_d \equiv [d_1^1, \ldots, d_p^1, \ldots, d_1^n, \ldots, d_p^n] \\ [e_1, \ldots, e_n] \Downarrow \langle [n, s_1, \ldots, s_m], v_d \rangle \end{array}}$$

For non-scalar values, there has to be a mapping between a multi-dimensional array and its flat representation. As a default mapping we use standard row-major order. This mapping is fixed for any SAC-λ program. In order to define the semantics of selections, we first formalise the mapping of index vectors within n-dimensional index spaces into offsets within the row-major representation of n-dimensional arrays:
$\mathcal{R}m :: \langle [n], [idx_1, \ldots, idx_n] \rangle \times \langle [n], [shape_1, \ldots, shape_n] \rangle \rightarrow \mathbb{N}$. $\mathcal{R}m$ is a bijective function which is defined as follows:

$$\mathcal{R}m(\langle [n], [i_1, \ldots, i_n] \rangle, \langle [n], [s_1, \ldots, s_n] \rangle) = \sum_{k=1}^{n} \left(\prod_{j=k+1}^{n} s_j \right) i_k$$

As $\mathcal{R}m$ is a bijective function, we can define $\mathcal{R}m^{-1}$ by integer division denoted as `div` and by modulo operations denoted as `mod`.

$$\mathcal{R}m^{-1}(\langle [], [a] \rangle, \langle [n], [s_1, \ldots, s_n] \rangle = \langle [n], [a_1, \ldots, a_n] \rangle$$

where

$$\forall k \in \{1, \ldots, n\} : a_k = (a \, \texttt{mod} \prod_{i=k}^{n} s_i) \, \texttt{div} \prod_{i=k+1}^{n} s_i.$$

With these definitions we obtain as a semantics for selections:

$$\text{SEL}: \frac{\begin{array}{c} iv \Downarrow \langle \, [n], [i_1, \ldots, i_n] \, \rangle \\ e \Downarrow \langle \, [s_1, \ldots, s_n], [d_1, \ldots, d_m] \, \rangle \end{array}}{\begin{array}{c} \texttt{sel (}iv\texttt{, } e\texttt{)} \Downarrow \langle \, [], [d_l] \, \rangle \\ \text{where } \langle [], [l] \rangle = \mathcal{R}m(iv, \langle [n], [s_1, \ldots, s_n] \rangle) + 1 \end{array}}$$

Please note that we add one to the row-major order as our enumeration of array elements in tuples starts with one while our row-major mapping assumes the C convention for indices, i.e., 0 is considered the lowest legal index.

Binary scalar arithmetic functions like addition, multiplication, etc. are defined on all the numerical types for scalar values only. Further down we demonstrate semantics of plus, which can be generalised to any primitive binary function by trivial substitution.

$$\texttt{+}: \frac{e_1 \Downarrow \langle \, [], [d_1] \, \rangle \quad e_2 \Downarrow \langle \, [], [d_2] \, \rangle}{e_1 \; \texttt{+} \; e_2 \Downarrow \langle \, [], [d_1 + d_2] \, \rangle}$$

Any scalar primitive function f has a built-in vector variant (denoted $\vec{f}$) with the following semantics:

$$\vec{f}(\vec{x}, \vec{y}) = [f(\vec{x}[0], \vec{y}[0]), \ldots, f(\vec{x}[V-1], \vec{y}[V-1])]$$

The layout type of $\vec{f}$ is $(\triangle, \triangle) \rightarrow \triangle$, which means that they are invisible to a user an can be obtained only by means of transformation.

We omit the semantics of LET, APP and IF as they are box-standard rules as they can be found in text books such as in [9].

The semantics for the data parallel operations MAP and REDUCE can be found at Fig 2. These two operations represent a simplified version of the with-loop construct of SAC. Both operators compute expressions over multidimensional index space; `map` combines the resulting expressions into an array and `reduce` folds them using a binary function. It is within the obligation of the programmer to ensure that the folding function is associative and commutative and that the expression provided as the last part of the `reduce` operation constitutes the neutral element of the folding function.

2.2 Layout inference

Layout types are used to hold information about the kind of vectorisation we apply for a given expression. For n-dimensional arrays we consider n different layout transformations (across each axis). The layout type representing such a vectorisation is denoted with natural numbers $1, \ldots, n$, where k refers to vectorisation across k-th axis. In this case the layout transformation changes the shape of an array from $[s_1, \ldots, s_n]$ to $[s_1, \ldots, \lceil {}^{s_k}/_{V} \rceil, \ldots, V]$. Layout type 0 refers to an unmodified value. All the expressions in the original program are of layout type 0.

We use layout type $\triangle$ to denote a stacking of V arrays of the original shape. In this case, the shape $[s_1, \ldots, s_n]$ transforms into $[s_1, \ldots, s_n, V]$. This layout type plays a key role in vectorisation of map/reduce operations together with $idx(k), k \in \mathbb{Z}_+$ layout type applied to index vectors.

We introduce layout types for functions to capture different layout transformations of an individual function. We denote that with $(l_1, \ldots, l_n) \rightarrow l$ for an n-argument function. For all l_i and l are in

$$\mathbb{N} \cup \{\triangle\} \cup idx(k), k \in \mathbb{Z}_+.$$

The essence of the layout transformation lies in replacement of the inner expression of map/reduce operations with the transformed

$$\text{MAP}: \frac{\begin{array}{c} e_u \Downarrow \langle\, [n], [u_1, \ldots, u_n] \,\rangle \\ \forall i_1 \in \{0, \ldots, u_1 - 1\} \ldots \forall i_n \in \{0, \ldots, u_n - 1\} : (\lambda\, iv.e_{op}\ \ [i_1, \ldots, i_n]) \\ \Downarrow \langle\, [s_1, \ldots, s_m], [d_1^{[i_1,\ldots,i_n]}, \ldots, d_p^{[i_1,\ldots,i_n]}] \,\rangle \end{array}}{\begin{array}{c} \texttt{map}\ iv < e_u\ e_{op} \\ \Downarrow \langle\, [u_1, \ldots, u_n, s_1, \ldots, s_m], [d_1^{[0,\ldots,0]}, \ldots, d_p^{[0,\ldots,0]}, \ldots, d_1^{[u_1-1,\ldots,u_n-1]}, \ldots, d_p^{[u_1-1,\ldots,u_n-1]}] \,\rangle \end{array}}$$

$$\text{REDUCE}: \frac{\begin{array}{c} e_u \Downarrow \langle\, [n], [u_1, \ldots, u_n] \,\rangle \qquad e_{neut} \Downarrow \langle\, [s_1, \ldots, s_m], [d_1, \ldots, d_p] \,\rangle \\ \forall i_1 \in \{0, \ldots, u_1 - 1\} \ldots \forall i_n \in \{0, \ldots, u_n - 1\} : (\lambda\, iv.e_{op}\ \ [i_1, \ldots, i_n]) \\ \Downarrow \mathfrak{D}^{[i_1,\ldots,i_n]} \equiv \langle\, [s_1, \ldots, s_m], [d_1^{[i_1,\ldots,i_n]}, \ldots, d_p^{[i_1,\ldots,i_n]}] \,\rangle \\ f(f(\ldots f(f(e_{neut}, e_{neut}), \mathfrak{D}^{[0,\ldots,0]}), \ldots), \mathfrak{D}^{[u_1-1,\ldots,u_n-1]}) \\ \Downarrow res \equiv \langle\, [s_1, \ldots, s_m], [r_1, \ldots, r_p] \,\rangle \end{array}}{\texttt{reduce}\ iv < e_u\ (f)\ e_{neut} \Downarrow res}$$

Figure 2. Semantics of map and reduce operations in SAC-λ

expression of layout type $\triangle$, preserving the original algorithm. For example, consider a map:

```
map i < [n,m] a[i] + b[i]
```

we can choose a vectorisation for the overall `map` construct by giving $idx(1)$ or $idx(2)$ layout type to `i` and replacing $+$ with $\vec{+}$, which is of layout type $(\triangle, \triangle) \to \triangle$. However, instead of $+$ we can have an arbitrary expression, so the $idx(k)$ layout types have to be propagated inside the inner expression of `map`, as these layout types introduce constraints on selections from arrays.

In order to make sure that the layout types are sound across the overall program, we use the layout type system, introduced in [12]. In the same work we sketch an inference algorithm to assign layout types to non-vectorised programs. We are not going to repeat all the rules here, however, we are going to use the fact that a certain expression can have a limited number of valid layout-types in the proof of correctness of the transformation.

2.3 Formal problem formulation

For every program F, after the layout inference is finished the new program F' is being generated, which is identical to F, but every expression e_i gets a new layout-type τ_i. After that the code in F' has to be transformed using transformation $\mathcal{T}$. The transformation adjusts iteration indexes, function applications, selections and map/reduce operations. Semantical correctness of the program under transformation $\mathcal{T}$ depends on the layout-type F' application returns. For layout-types $k \in \mathbb{Z}_+$ we require per-element identity of F and F' application. This identity holds under some index mapping between an array of layout type 0 and of layout type k. We denote this mapping with I_k and require it to be bijective. In terms of this paper we consider one particular mapping which is defined as follows:

$$\begin{aligned} &I_k(\langle [n], [i_1, \ldots, i_n] \rangle) \\ &\quad = \langle [n+1], [i_1, \ldots, i_k \ \texttt{div}\ V, \ldots, i_n, i_k \ \texttt{mod}\ V] \rangle \end{aligned}$$

and the inverse variant of the same function:

$$\begin{aligned} &I_k^{-1}(\langle [n], [i_1, \ldots, i_n] \rangle) \\ &\quad = \langle [n-1], [i_1, \ldots, Vi_k + i_n, \ldots i_{n-1}] \rangle \end{aligned}$$

For the $\triangle$ types we cannot use the same criterion from above, as F' actually might return different values, which cannot be related to the original input. However, we can check that algorithm-wise the application of F' is the same as V applications of F.

Finally, for $idx(k)$ types we require that $F\{F_k \to F_k/V\}$ (application of F, where k-th component is substituted with its original value divided by V) is equal to F'.

We introduce two concatenation operations which we use in the definition of correctness and subsequent proofs. The standard concatenation is defined as a binary operation and denoted with a symbol $+\!\!+$. Shape-wise $+\!\!+$ concatenates two arrays extending the first component of the shape. The following semantic rule defines the operation:

$$+\!\!+ : \frac{\begin{array}{c} e_1 \Downarrow \langle\, [s_1, s_2, \ldots, s_n], [d_1, \ldots, d_p] \,\rangle \\ e_2 \Downarrow \langle\, [s'_1, s_2, \ldots s_n], [d'_1, \ldots, d'_q] \,\rangle \\ s \equiv [s_1 + s'_1, s_2, \ldots, s_n] \end{array}}{e_1 +\!\!+ e_2 \Downarrow \langle\, \mathrm{s}, [d_1, \ldots, d_p, d'_1, \ldots, d'_q] \,\rangle}$$

The $+\!\!+$ would work as follows: $[1, 2, 3] +\!\!+ [3, 4] = [1, 2, 3, 4]$.

Another operation is called array stacking, denoted with $\oplus$. This is an n-ary operation defined using the following rule:

$$\oplus : \frac{\begin{array}{c} e_1 \Downarrow \langle\, [s_1, s_2, \ldots, s_m], [d_1^1, \ldots, d_p^1] \,\rangle \\ \ldots \\ e_n \Downarrow \langle\, [s'_1, s_2, \ldots s_m], [d_1^n, \ldots, d_p^n] \,\rangle \\ s \equiv [s_1, \ldots, s_m, n] \end{array}}{\oplus(e_1, \ldots, e_n) \Downarrow \langle\, \mathrm{s}, [d_1^1, \ldots d_1^n \ldots, d_p^1, \ldots, d_p^m] \,\rangle}$$

The application of $\oplus$ works as follows: $\oplus([1, 2, 3], [4, 5, 6]) = [[1, 4], [2, 5], [3, 6]]$. It works similarly to the concatenation, but adds an extra dimension and does concatenation on the last shape component rather than on the first one. In the rest of the paper we are going to use the following notation: $\oplus_{i=0}^{n} f(i)$ which is a shortcut for $\oplus(f(0), \ldots, f(n-1))$.

In order to define correctness of the transformation formally, we introduce a symbol $\mathcal{C}$ which has a co-domain $\{T, F\}$ (true or false). We say that expression e is correct under transformation T denoted as $\mathcal{C}(\mathcal{T}, e) \equiv T$ if:

$$\mathcal{C}(\mathcal{T}, e) \Leftrightarrow \langle a_1, \ldots, a_n \rangle = FV(e) \wedge \exists f(a_1, \ldots, a_n) = e$$

where $FV(e)$ are the free variables of the expression e and if the layout-type of $\mathcal{T}(f)$ is $(\ldots) \to \triangle$, then

$$\begin{aligned} &\oplus_{j=0}^{V} f(\mathcal{T}^{-1}(\mathcal{T}(a_1), j), \ldots, \mathcal{T}^{-1}(\mathcal{T}(a_n), j) \\ &\quad \equiv \mathcal{T}(f)(\mathcal{T}(a_1), \ldots, \mathcal{T}(a_n)) \end{aligned}$$

where

$$\mathcal{T}^{-1}(x, j) = \begin{cases} x & x :: 0 \\ x' | x'[I_k^{-1}(i)] = x[i] & x :: k \\ x\{x_k \to Vx_k + j\} & x :: idx(k) \\ x[*, j] & x :: \triangle \end{cases}$$

if the type of $\mathcal{T}(f)$ is $(\ldots) \to k, k \in \mathbb{N}$ then

$$\forall i < s_f : f(a_1, \ldots, a_n)[i] \stackrel{?}{\equiv} \mathcal{T}(f)(\mathcal{T}(a_1), \ldots, \mathcal{T}(a_n))[I_k(i)];$$

and if the type of $\mathcal{T}(f)$ is $(\ldots) \to idx(k), k \in \mathbb{Z}_+$, then

$$f(a_1, \ldots, a_n)\{f_k \to f_k/V\} \equiv \mathcal{T}(f)(\mathcal{T}(a_1), \ldots, \mathcal{T}(a_n))$$

In the case of k layout type we use $\forall_{i<s_e}$ quantification which reads as: for all indexes starting from $[0, \ldots, 0]$ and ending with the shape of expression e. We are going to use this notation through the rest of the paper.

The expression $e[*, j]$ used in $\mathcal{T}^{-1}$ definition means an array slice with the last index component fixed at j. In essence this is a reverse operation for $\oplus$ which allows to grab j-th component. Semantically it looks like this:

$$\text{SLICE}: \frac{\begin{array}{c} e \Downarrow \langle\, [s_1, \ldots, s_{n-1}, s_n], [d_1, \ldots, d_{ps_n}] \,\rangle \\ iv \Downarrow \langle\, [], [j] \,\rangle \wedge j < s_n \\ s \equiv [s_1, \ldots, s_{n-1}] \end{array}}{e[*, iv] \Downarrow \langle\, \mathrm{s}, [d_{j+1}, d_{s_n+j+1}, \ldots, d_{(p-1)s_n+j+1}] \,\rangle}$$

For example, $([[1, 2, 3], [4, 5, 6]])[*, 1] = [2, 5]$.

3. Program transformation

It is easy to see that if the layout inference succeeded then LET term can be used as is. APP term has to make sure that it picks the right function from the environment, which is a matter of layout-type signature matching. IF term has to be transformed in case its condition is of lyout type $\triangle$. In that case *true* branch and *false* branch have to be evaluated and the results have to be masked using the evaluated condition. This is a standard control-flow to data-flow transformation widely discussed in [5], however, as a functional setup does not have control flow, most of the problems are not relevant in the given setting. Essential parts regarding the transformation are CONST, SEL, PRF, MAP and REDUCE.

Informally speaking, array vectorisation in terms of the given work is a cutting of an array over axis τ using tiles of size $1 \times V$ and putting V-sized chunks in such a way that the elements become adjacent in flattened representation (in our cases under $\mathcal{R}m$). Please note, that by doing so we potentially increase the number of elements in the vectorised array, as the size of τ axis might be not divisible by V. This would be a problem if we would ever make a reverse array vectorisation with further selection at newly introduced indexes. We never do that, and we never shuffle elements of the vector, so the new "phantom" elements of the array never participate in real computations. The only thing that we should show is that index translations within the program stay sound.

Array vectorisation presented by I_τ can be done differently, which would have an impact on the further transformation. For example one could split arrays into two parts, using floor operation instead of ceiling, or permuting the vectorised axis, however, in this paper we will fix it to the one we find most practically relevant.

3.1 CONST

A constant c can get the following layout types: 0, in which case the constant stays the same as in the original program; $1, \ldots, \mathcal{D}(C)$, in which case the data is being vectorised and $\triangle$, in which case the constant is being stacked V times.

Constant transformation at $\tau \in \mathbb{Z}_+$ is cutting an array at axis τ and puts V adjacent elements of the τ axes in a such a way that they are identical in the flattened row-major representation.

$$\begin{aligned} & s_c \equiv [s_1, \ldots, s_n] \\ & s_{c'} \equiv [s_1, \ldots, \lceil {}^{s_\tau}/_V \rceil, \ldots, s_n, V] \\ & c \equiv \langle s_c, [d_1, \ldots, d_p] \rangle \\ & c' \equiv \langle s_{c'}, [d'_1, \ldots, d'_{p'}] \rangle \\ & \mathcal{T}(c) = c' \end{aligned}$$

where

$$\begin{aligned} & \forall k \in \{1, \ldots, \prod_{j=1}^{n+1} s_{c'}[j]\} : \\ & \quad cidx' \equiv \mathcal{R}m^{-1}(k-1, s_{c'}) \\ & \quad zero' \equiv [1, \ldots, 1, 0] \\ & \quad d'_k = \begin{cases} d_{\mathcal{R}m(I_\tau^{-1}(cidx'), s_c)+1} & I_\tau^{-1}(cidx) < s_c \\ d_{\mathcal{R}m(I_\tau^{-1}(cidx' \cdot zero'), s_c)+1} & \text{otherwise} \end{cases} \end{aligned}$$

when $\tau \in 1, \ldots, \mathcal{D}(c)$

In this case multiplication and less than operators are used per vector component. Please note the way we treat "phantom" elements of the vectorised array – we fill the empty cells with the first element of each vector, which should be always present, as zero axes sizes are not supported. As a result, any vectorised operation on such a vector succeeds, if according scalar operation succeeds in the original program. Otherwise, if we were using constant elements instead, it would be easy to construct an example resulting in division by zero or arithmetic overflow, which might cause failures during program execution.

Finally we consider the case when c is of layout-type $\triangle$ which means that the array has to be stacked V times.

$$\mathcal{T}(c) = \oplus_{i=0}^{V} c$$

3.2 SEL

Selections can manifest themselves in three different ways: scalar selection, vector selection and scalar selection on a vectorised array. The scalar case is applicable when the expression we are selecting from is of layout-type 0, for example in `sel`$(iv, a), a :: 0$.

Vector selections can happen in two cases:

1. $a :: k \wedge iv :: idx(k)$, which means that this selection is invoked within a map/reduce operation, and index space is vectorised over the k-th axis. As an example consider a map over 1-d array where the inner operation uses selection:

   ```
   map i < [N] f (a[i])
   ```

   in which case if $i :: idx(1) \wedge a :: 1$, the selection has to return an expression of vector layout type rather than of scalar;

2. $a :: \triangle \wedge iv :: 0$ – this case can be found in nested maps, when selection happens on the axis which was not vectorised. For example, assume we have 2-d array a of shape $[M, N]$:

   ```
   map i < [M]
       let
           line = map j < [N] a[i ++ j]
       in
           f (line[iv])
   ```

   in this case, assuming that $i :: idx(1)$, j can be only of type 0, and the $line$ has to become a 1-d array of vectors rather than of scalars, which means that $line$ gets a layout type $\triangle$. Any

further selections on $line$ implies vector selections with index of layout type 0.

Semantically, vector selection is a concatenation of V scalar selections. Keep in mind, that the length of an index vector in this cases is one element shorter than the dimensionality of the vectorised array.

$$\text{VSEL}: \frac{\begin{array}{c} iv \Downarrow \langle\, [n-1], [i_1, \ldots, i_{n-1}]\, \rangle \\ e \Downarrow \langle\, \langle [n], [s_1, \ldots, s_{n-1}, V] \rangle, [d_1, \ldots, d_m]\, \rangle \end{array}}{\begin{array}{c} \texttt{vsel}\ (iv,\ e) \Downarrow \langle\, [\mathrm{V}], [d'_1, \ldots, d'_i]\, \rangle \\ \text{where } \forall i \in \{1, \ldots, V\}: \\ \langle [], [d'_i] \rangle = \texttt{sel}(iv +\!\!+ [i-1], e) \end{array}}$$

Finally, when selections on a vectorised array are performed outside the map/reduce operation, the result of the selection has to be scalar. To illustrate that assume a data-parallel operation on the array a after which one scalar element is being modified:

```
let
    r = map i < u f(a[i])
in
    g (r[e])
```

Assuming that $a :: k \wedge r :: k \wedge e :: 0$, we adopt index e by applying I_k index transformation.

Summarising transformations together, we get the following expression:

$$\mathcal{T}(\texttt{sel}(iv, a)) = \begin{cases} \texttt{sel}(iv, a) & iv :: 0 \wedge a :: 0 \\ \texttt{vsel}(iv, a) & iv :: idx(k) \wedge a :: k \\ \texttt{vsel}(iv, a) & iv :: 0 \wedge a :: \triangle \\ \texttt{sel}(I_k(iv), a) & iv :: 0 \wedge a :: k \end{cases}$$

3.3 Primitive functions PRF

All the primitive scalar operations have built-in vector variants, so the only thing the transformation should take care of is vectorisation of the scalar component, which is allowed by the layout-type system. Again, without lose of generality we consider plus being a representative of all primitive functions.

$$\mathcal{T}(a+b) = \begin{cases} a+b & a :: 0 \wedge b :: 0 \\ \oplus_{i=0}^{V} a \vec{+} b & a :: 0 \wedge b :: \triangle \\ a \vec{+} \oplus_{i=0}^{V} b & a :: \triangle \wedge b :: 0 \\ a \vec{+} b & a :: \triangle \wedge b :: \triangle \end{cases}$$

3.4 MAP

The only case when a map construct requires code transformation is when its index is of layout-type $idx(k)$ and its expression is of layout-type $\triangle$. In this case the upper bound will be transformed as follows:

$$\begin{aligned} &\mathcal{T}(\texttt{map}\ j < \langle [n], [u_1, \ldots, u_k, \ldots, u_n] \rangle\ e) \\ &\quad = \texttt{map}\ j < \langle [n], [u_1, \ldots, \lceil u_k/V \rceil, \ldots, u_n] \rangle\ \mathcal{T}(e) \end{aligned}$$

We apply I_k to u, but we get rid of the last vector component, which makes sure that selection at the index variable j would require an expression we are selecting from to be of type k.

In all the other cases:

$$\mathcal{T}(\texttt{map}\ j < u\ e) = \texttt{map}\ j < \mathcal{T}(u)\ \mathcal{T}(e)$$

3.5 REDUCE

The only case when `reduce` requires a transformation is, similarly to `map`, when the layout type of the index vector is $idx(k)$. The inner expression of `reduce` would be of layout type $\triangle$. The upper bound expression has to be transformed in the same way as we do in `map`, however, in contrast to `map`, the resulting layout type of `reduce` has be 0. It does not happen automatically as a return layout type of a reduction function is $\triangle$, which implies an extra step of reduction. First we consider the case when the axis of vectorisation is divisible by V i.e. there are no "phantom" elements:

$$\begin{aligned} &\mathcal{T}(\texttt{reduce}\ i < \langle [n], [u_1, \ldots, u_k, \ldots, u_n] \rangle\ (f)\ e) = \\ &\texttt{let} \\ &\quad \vec{r} = \texttt{reduce} \\ &\qquad\quad i < \langle [n], [u_1, \ldots, {}^{u_k}/_V, \ldots, u_n] \rangle\ (\mathcal{T}(f))\ \mathcal{T}(e) \\ &\texttt{in} \\ &\quad \texttt{reduce}\ i < [V]\ (f)\ \texttt{r[*,}i\texttt{]} \end{aligned}$$

In order to see this transformation in action, consider a summation (with +) of a 1-d array a of shape $[NV]$:

$$\texttt{reduce}\ i < [N \cdot V]\ (+)\ \texttt{a[}i\texttt{]}$$

Assuming that a has layout type 1, the following vectorisation

$$\vec{r} \equiv \texttt{reduce}\ i < [N]\ (\vec{+})\ \texttt{a[}i\texttt{]}$$

would deliver a result of shape $[V]$ which has to be reduced with a scalar plus:

$$\texttt{reduce}\ i < [V]\ (+)\ \vec{r}[i]$$

Now let us consider the case when the size of the vectorisation axis is not divisible by V. Two things to notice here: first, we cannot perform a vector operation on the last vector (an element of vectorised array which has $\lceil s_k/V \rceil$ at its k-th position in the index), as it would contain "phantom" elements. For example, if $V \equiv 4$ then vectorised vector $[1, 2, 3, 4, 5, 6, 7, 8, 9]$ will look like $[[1, 2, 3, 4], [5, 6, 7, 8], [9, x, x, x]], x = 9$, and we cannot add the last vector, but we can still add first two. Using this observation we split the index space of vectorised reduce into two parts: $[0, \ldots, 0] \leq i < [u_1, \ldots, \lfloor u_k/V \rfloor, \ldots, u_n]$ and $[0, \ldots, \lfloor u_k/V \rfloor, 0] \leq i < [u_1, \ldots, \lceil u_k/V \rceil, \ldots, u_n]$. Formally we denote that as follows:

$$\begin{aligned} &\mathcal{T}(\texttt{reduce}\ i < \langle [n], [u_1, \ldots, u_k, \ldots, u_n] \rangle\ (f)\ e) = \\ &\texttt{let} \\ &\quad \vec{r} = \texttt{reduce} \\ &\qquad\quad i < \langle [n], [u_1, \ldots, \lceil ({}^{u_k}/_V - \lfloor {}^{u_k}/V \rfloor)/V \rceil, \ldots, u_n] \rangle \\ &\qquad\quad (\mathcal{T}(f))\ \mathcal{T}(e); \\ &\quad \vec{r'} = \texttt{reduce} \\ &\qquad\quad i' < u' \equiv \langle [n], [u_1, \ldots, \lceil u_k/V \rceil, \ldots, u_n] \rangle \\ &\qquad\quad (\mathcal{T}(f)) \\ &\qquad\quad \texttt{let} \\ &\qquad\qquad i = \langle [n], [i'_1, \ldots, i'_k + \lfloor u_k/V \rfloor, \ldots, i'_n] \rangle \\ &\qquad\quad \texttt{in} \\ &\qquad\qquad \mathcal{T}(e); \\ &\texttt{in} \\ &\quad \texttt{let} \\ &\qquad r_1 = \texttt{reduce}\ i < [V]\ (f)\ \vec{r}\texttt{[*, }i\texttt{]}; \\ &\qquad r_2 = \texttt{reduce}\ i < [u_k\ mod\ V]\ (f)\ \vec{r'}\texttt{[*,}i\texttt{]} \\ &\quad \texttt{in} \\ &\qquad f(r_1, r_2) \end{aligned}$$

In order to illustrate the final formula, consider a reduction with plus over an array of shape $[10, N]$ being of layout-type 1. In this case $\vec{r}$ would hold a result of vectorised addition of vectors $[0, *, *]$ and $[1, *, *]$. Then $\vec{r'}$ would sum-up vectors $[2, *, *]$, using vector plus. Please note that we can do this, as operation on "phantom" elements would not affect the result. Finally, r_1 would sum-up the elements inside the vector and r_2 would sum-up non-phantom elements of $\vec{r'}$.

Finally, in case when reduce is of layout type $\triangle$ or k the transformation is defined as:

$\mathcal{T}$(`reduce` i `<` u `(`f`)` e`)` $=$ `reduce` i `<` u `(`$\mathcal{T}(f)$`)` $\mathcal{T}(e)$

3.6 IF

Transformation of a conditional requires control-flow to data-flow transformation with potential stacking of a non-vectorised branch. Formally we denote it as follows:

$\mathcal{T}$(`if` e_1 `then` e_2 `else` e_3`) =`
e_2' `=` $\oplus_{i=0}^{V} e_2$ if $e_2 :: 0 \wedge e_3 :: \triangle$ `or` $\mathcal{T}(e_2)$ `otherwise`
e_3' `=` $\oplus_{i=0}^{V} e_3$ if $e_3 :: 0 \wedge e_2 :: \triangle$ `or` $\mathcal{T}(e_3)$ `otherwise`
`let`
  b `=` $\mathcal{T}(e_1)$`;`
`in`
  `mask (`t`,` e_2'`,` e_3'`)`

for the cases when $e_1 :: \triangle$.

The `mask` operation chooses either from the second or from the third argument depending on the boolean value in the first argument:

`mask (`m`,` t`,` f`)`
  `=` $\oplus_{i=0}^{V}$ `(if` $m[i]$ `then` t`[*,`i`]` `else` f`[*,i])`

Real implementation of `mask` might differ from the code from above, as some of the architectures have either built-in masking facilities for 1-d vectors, or can be implemented efficiently using vector binary operations like bitwise and, bitwise or and bitwise not. Semantically it does exactly the same thing.

3.7 APP, LET and LETREC

The transformation of these three expression types is pretty much straight forward, we just propagate $\mathcal{T}$ to the arguments, the only important moment here is the way function definitions are being stored.

Vectorisation of a function definition boils down to vectorisation of its body:

$\mathcal{T}$(`f`$(a_1, \ldots, a_n)$ `=` e`)` `=` `f`$(a_1, \ldots, a_n)$ `=` $\mathcal{T}(e)$

However, the vectorised function does not replace the original one, but it rather creates a new instance with the same name but different layout-type signature. It means that we have a family of functions, which can be differentiated by SAC-λ using overloading mechanisms with respect to layout-types.

The application of a function boils down to matching the right function overloading according to its layout-type signature. The matching is trivial so we would assume that the compiler does it under the hood. The transformation for the application is defined as follows:

$\mathcal{T}(f(e))$ `=` $f(\mathcal{T}(e)$

The same reasoning is applicable to let constructs:

$\mathcal{T}$(`let x =` e_1 `in` e_2`)` `= let` x `=` $\mathcal{T}(e_1)$ `in` $\mathcal{T}(e2)$

Finally the letrec construct is a vectorisation of its goal expression and all the fundefs according to the new layout-type signatures.

$\mathcal{T}$(`letrec` $f_1(\ldots) = e_1, \ldots f_n(\ldots) = e_n$ `in` e`)`
  `=letrec` $\mathcal{T}(f_1(\ldots) = e_1), \ldots \mathcal{T}(f_n(\ldots) = e_n)$ `in` $\mathcal{T}(e)$

4. Semantical correctness

In this section we are going to construct a proof that any program in SAC-λ transformed by $\mathcal{T}$ is semantically correct in terms of $\mathcal{C}$. We are going to use structural induction over terms of the language.

First of all let us start with simple general observations.

Theorem 1. *For a given function $f(a_1, \ldots, a_n)$ and transformation $\mathcal{T}(f) :: (0, \ldots, 0) \to 0$, $\mathcal{C}(\mathcal{T}, f) \equiv T$.*

Proof. Obvious due to identity. □

The second observation concerns the $idx(k)$ expressions.

Theorem 2. *For a given expression e, where $\mathcal{T}(e) :: idx(k)$, $\mathcal{C}(\mathcal{T}, e) \equiv T$.*

Proof. By definition of $\mathcal{C}$ we assume that there is a function with arguments $FV(e)$, body e and return type $idx(k)$. We need to show that the application of $\mathcal{T}(f)$ to $\mathcal{T}(FV(e))$ results in the same value as application of f to $FV(e)$. Now, according to the layout-type system, we conclude that if a function returns a value of type $idx(k)$ then there is an argument in $iv \in \mathcal{T}(FV(e))$ of type $idx(m)$ and only two cases are possible:

1. *propagation,* $k = m$ which means that the function returns iv, in which case the correctness is obvious;
2. *concatenation* which means that iv is a part of $+\!\!+$ operation. If the operation is $iv +\!\!+ e$, then assuming that iv is correct, the k-th component of the expression would be identical to the k-th component of iv, which makes resulting expression correct. Alternatively, the expression could be $e +\!\!+ iv$, where $e :: 0$ and by construction of $+\!\!+$ the k-th component of the resulting expression is mapped to m-th component of iv. That makes resulting expression correct.

□

We deliberately restrict the range of operations on index vectors for the time being, so the proof simply runs structural induction for this types.

The base case of the structural induction is a constant, as we do not allow external parameters, and all the values are conceptually defined by means of constants.

Theorem 3. *Constant transformation is semantically correct.*

$$\frac{c \text{ is } \texttt{const}}{\mathcal{C}(\mathcal{T}, c) \stackrel{?}{\equiv} T}$$

Proof. By the construction of CONST transformation. Let us consider three possible cases of the layout-type of a transformed constant:

$\mathcal{T}(c) :: 0$ non-vectorised case, obvious.

$\mathcal{T}(c) :: \triangle$ in which case we show that

$$\oplus_{i=0}^{V} \mathcal{T}(c)[*, i] \stackrel{?}{\equiv} \mathcal{T}(c)$$

which is obvious as $\mathcal{T}(c) \equiv \oplus_{i=0}^{V} c$. By the way, this is the only case for expressions of layout type $\triangle$ when the scalar elements of the transformed one can be mapped to the scalar elements of the original one.

$\mathcal{T}(c) :: k \in \mathbb{Z}_+$ in which case we show that

$$\forall_{i<s_c} : c[i] \stackrel{?}{\equiv} \mathcal{T}(c)[I_k(i)]$$

This follows from the construction of $\mathcal{T}(c)$, and the phantom elements are not in the range of i.

□

Theorem 4. *Vectorisation of selection is semantically correct.*

$$\frac{e \equiv a[iv] \quad \mathcal{C}(\mathcal{T}, a) \equiv T \quad \mathcal{C}(\mathcal{T}, iv) \equiv T}{\mathcal{C}(\mathcal{T}, e) \stackrel{?}{\equiv} T}$$

Proof. Let us consider all the legal layout types for transformed e, iv and a:

$\mathcal{T}(e) :: 0 \wedge \mathcal{T}(a) :: 0 \wedge \mathcal{T}(iv) :: 0$ non-vectorised, obvious.

$\mathcal{T}(e) :: \triangle \wedge \mathcal{T}(a) :: k \wedge \mathcal{T}(iv) :: idx(k)$ in which case we need to show that:

$$\oplus_{j=0}^{V} \texttt{sel}(\mathcal{T}(iv)\{\mathcal{T}(iv)_k \to V\mathcal{T}(iv)_k + j\}, \mathcal{T}^{-1}(\mathcal{T}(a)) \equiv \texttt{vsel}(\mathcal{T}(iv), \mathcal{T}(a))$$

From the construction of `vsel` we construct the following equality:

$$\texttt{vsel}(\mathcal{T}(iv), \mathcal{T}(a)) \equiv \oplus_{j=0}^{V}\texttt{sel}(\mathcal{T}(iv) +\!\!+ [j], \mathcal{T}(a))$$

Now, from $\mathcal{T}^{-1}$ construction for layout type k, we get the following mapping between $\mathcal{T}(a)$ and $\mathcal{T}^{-1}\mathcal{T}(a)$:

$$\forall_{i<s_{\mathcal{T}(a)}} \mathcal{T}^{-1}(\mathcal{T}(a))[I_k^{-1}(i)] \equiv \mathcal{T}(a)[i]$$

From $\mathcal{C}(\mathcal{T}, a)$ the shape of $\mathcal{T}(a)$ is $[s_1, \ldots, s_n, V]$. The shape of $\mathcal{T}(iv)$ is $[s_1, \ldots, s_n]$, which allows to rewrite the original expression to:

$$\oplus_{j=0}^{V} \texttt{sel}(\mathcal{T}(iv)\{\mathcal{T}(iv)_k \to V\mathcal{T}(iv)_k + j\}, \mathcal{T}^{-1}(\mathcal{T}(a)) \equiv \oplus_{j=0}^{V}\texttt{sel}(\mathcal{T}(iv) +\!\!+ [j], \mathcal{T}(a))$$

which is obvious due to construction of I_k^{-1}.

$\mathcal{T}(e) :: \triangle \wedge \mathcal{T}(a) :: \triangle \wedge \mathcal{T}(iv) :: 0$ in which case we need to show that

$$\oplus_{j=0}^{V} \texttt{sel}(iv, \mathcal{T}(a)[*, j]) \equiv \texttt{vsel}(iv, \mathcal{T}(a))$$

From $\mathcal{C}(\mathcal{T}, a)$ we deduce that $\mathcal{T}(a) = \oplus_{i=0}^{v}\mathcal{T}(a)[*, i]$ and from the construction of `vsel`, correctness becomes obvious.

$\mathcal{T}(e) :: 0 \wedge \mathcal{T}(a) :: k \in \mathbb{Z}_+ \wedge \mathcal{T}(iv) :: 0$ in which case we need to show that

$$\texttt{sel}(iv, a) \equiv \texttt{sel}(I_k(iv), \mathcal{T}(a))$$

which directly follows from $\mathcal{C}(\mathcal{T}, a)$.

□

Theorem 5. *Vectorisation of a primitive function is semantically correct.*

$$\frac{e \equiv x + y \quad \mathcal{C}(\mathcal{T}, x) \equiv T \quad \mathcal{C}(\mathcal{T}, y) \equiv T}{\mathcal{C}(\mathcal{T}, e) \stackrel{?}{\equiv} T}$$

Proof. There are four possible layout-type combinations for a given expression.

$\mathcal{T}(e) :: 0 \wedge \mathcal{T}(x) :: 0 \wedge \mathcal{T}(y) :: 0$ which is obvious.

$\mathcal{T}(e) :: \triangle \wedge \mathcal{T}(x) :: \triangle \wedge \mathcal{T}(y) :: 0$ in which case we need to show that:

$$\oplus_{j=0}^{V} (\mathcal{T}(x)[j] + (\oplus_{i=0}^{V} y)[j]) \equiv \mathcal{T}(x) \vec{+} \oplus_{i=0}^{V} y$$

Please note that $\mathcal{T}(x)$ has a shape $[V]$ so we can replace slice operation with a simple selection. Now, from $\mathcal{C}(\mathcal{T}, x)$ we deduce that $\mathcal{T}(x) = \oplus_{i=0}^{V}\mathcal{T}(x)[i]$. Finally, as

$$a \vec{+} b \stackrel{def}{\equiv} [a[0] + b[0], \ldots, a[V-1] + b[V-1]] \equiv \oplus_{i=0}^{V} a[i] + b[i]$$

as a and b are of shape $[V]$. This transformation makes original equality obvious.

$\mathcal{T}(e) :: \triangle \wedge \mathcal{T}(x) :: 0 \wedge \mathcal{T}(y) :: \triangle$ same reasoning as in the previous case.

$\mathcal{T}(e) :: \triangle \wedge \mathcal{T}(x) :: \triangle \wedge \mathcal{T}(y) :: \triangle$ in which case $\mathcal{C}(\mathcal{T}, a) \implies \mathcal{T}(a) = \oplus_{i=0}^{V}\mathcal{T}(a)[i]$ and $\mathcal{C}(\mathcal{T}, b) \implies \mathcal{T}(b) = \oplus_{i=0}^{V}\mathcal{T}(b)[i]$ in which case we can use the same reasoning as in previous cases.

□

Theorem 6. *Vectorisation of a map is semantically correct:*

$$\frac{e \equiv \texttt{map}\ i < u\ e_1 \quad \mathcal{C}(\mathcal{T}, e_1) \equiv T}{\mathcal{C}(\mathcal{T}, e) \stackrel{?}{\equiv} T}$$

Proof. As e_1 is correct there is a function f with body e_1. Let us assume, without loss of generality, that f arguments have i at the first position. In case it is not a free variable of e_1 this argument will be skipped, but we can still pass it. Let us consider legal layout-type combinations of transformed e, i and f.

$\mathcal{T}(e) :: 0 \wedge \mathcal{T}(i) :: 0 \wedge \mathcal{T}(f) :: (0) \to 0$ non-vectorised case, obvious;

$\mathcal{T}(e) :: k \wedge \mathcal{T}(i) :: idx(k) \wedge \mathcal{T}(f) :: (idx(k)) \to \triangle$ Let us assume that the shape of non-transformed e_1 is scalar. Later we will demonstrate that the reasoning works for non-scalars as well. We need to show that:

$$\forall_{j<s_e}(\texttt{map}\ i < u\ f(i, \ldots))[j] \stackrel{?}{\equiv} (\texttt{map}\ i' < u\{u_k \to \lceil u_k/V \rceil\}\ \mathcal{T}(f)(i', \ldots))[I_k(j)]$$

From the correctness of f we have

$$\mathcal{T}(f)(\mathcal{T}(i), \ldots) \equiv \oplus_{j=1}^{V} f(\mathcal{T}(i)\{\mathcal{T}(i)_k \to V\mathcal{T}(i)_k + j\}, \ldots)$$

From the construction of I_k follows that

$$I_k(j) = j\{j_k \to j_k\ \texttt{div}\ V\} +\!\!+ [j_k\ \texttt{mod}\ V]$$

As we assumed that shape of f is scalar, let us note that the index space of j under the quantifier and i in the first map are the same. The index space of $j\{j_k \to j_k\ \texttt{div}\ V\}$ and the index space of i' us the same as well. The shape of $\mathcal{T}(f)$ is $[V]$, and we can rewrite the original expression as:

$$\forall_{j<s_e} f(j, \ldots)) \stackrel{?}{\equiv} \mathcal{T}(f)(j\{j_k \to j_k\ \texttt{div}\ V\}, \ldots))[j_k\ \texttt{mod}\ V]$$

which is obvious due to correctness of f.

$\mathcal{T}(e) :: \triangle \wedge \mathcal{T}(i) :: 0 \wedge \mathcal{T}(f) :: (0) \to \triangle$ in which case we need to show that

$$\oplus_{j=0}^{V} \mathcal{T}(\texttt{map}\ i < u\ \mathcal{T}(e_1)[*, j]) \equiv \texttt{map}\ i < u\ \mathcal{T}(e_1)$$

As $\mathcal{T}(e_1) :: \triangle$ and $\mathcal{C}(\mathcal{T}, e_1) \implies \oplus_{j=0}^{V}\mathcal{T}(e_1)[*, j] \equiv \mathcal{T}(e_1)$ the correctness of expression is obvious due to semantics of `map`.

$\mathcal{T}(e) :: \mathcal{D}(i) + k \wedge \mathcal{T}(i) :: 0 \wedge \mathcal{T}(f) :: (0) \to k \in \mathbb{Z}_+$ In this case we need to show that

$$\forall_{j<s_e}(\texttt{map}\ i < u\ e_1)[j] \stackrel{?}{\equiv} (\texttt{map}\ i < u\ \mathcal{T}(e_1))[I_{\mathcal{D}(i)+k}(j)]$$

From $\mathcal{C}(\mathcal{T}, e_1)$ follows that $\forall i : \forall j < s_{e_1} e_1[j] = \mathcal{T}(e_1)[I_k(j)]$ and the fact we prove reduces to this condition if for every index we drop $\mathcal{D}(i)$ first elements.

Finally we have to make sure that if e_1 is not scalar, the correctness still holds. Assuming that the shape of e_1 is s_{e_1} we would apply the same reasoning considering that all the values in our program are defined as $\langle [s_1, \ldots, s_n], [d_1, \ldots, d_p] \rangle$, where d_i is of shape s_{e_1}. That reduces non-scalar map to the scalar one, and makes the same reasoning applicable.

□

Theorem 7. *Vectorisation of reduce is semantically correct:*

$$\frac{\mathcal{C}(\mathcal{T}, f_{bin}) \equiv T \qquad \mathcal{C}(\mathcal{T}, e_1) \equiv T \qquad e \equiv \texttt{reduce}\ i\ \texttt{<}\ u\ (f_{bin})\ e_1}{\mathcal{C}(\mathcal{T}, e) \stackrel{?}{\equiv} T}$$

Similarly to map, due to $\mathcal{C}(\mathcal{T}, e_1)$ there exists a function f for which without lose of generality we introduce an argument i at the first position.

First of all let's consider the transformation informally, proving the properties we will use later. First observation is that a reduce operation can be considered as the following expression:

$$r = w_{i_1} \times w_{i_2} \times \cdots \times w_{i_n}$$

where w_k is a result of application of function $f(i, \ldots)$ to $(k, \ldots)$ and $\times$ is f_{bin} written in the infix notation.

First of all, as $\times$ is associative ($a \times (b \times c) \equiv (a \times b) \times c$, the original expression can be divided into two subgroups at index k where each subgroup can be computed concurrently:

$$\begin{aligned} r_1 &= (w_{i_1} \times \cdots \times w_k) \\ r_2 &= (w_{k+1} \times \cdots \times w_{i_n}) \\ r &= r_1 \times r_2 \end{aligned}$$

The fact that $\times$ is commutative ($a \times b \equiv b \times a$) allows us to permute elements e_k in the r expression:

$$r = w_{\psi(i_1)} \times w_{\psi(i_2)} \times \cdots \times w_{\psi(i_n)}$$

where $\psi(k)$ is a valid permutation in the range of $\{i_1, \ldots, i_n\}$.

Now let us consider what happens when vectorising r with $\vec{\times}$:

$$\begin{aligned} &[w'_1 \ldots, w'_{V-1}] \vec{\times} [w''_1, \ldots, w''_{V-1}] \\ &\quad \equiv [w'_1 \times w''_1, \ldots, w'_{V-1} \times w''_{V-1}] \end{aligned}$$

as follows:

$$\begin{aligned} &\vec{r} = [w_1^{(1)}, \ldots, w_{V-1}^{(1)}] \vec{\times} \cdots \vec{\times} [w_1^{(n/V)}, \ldots, w_{V-1}^{(n/V)}] \\ &r = \vec{r}[0] \times \cdots \times \vec{r}[V-1] \end{aligned}$$

We assume here that $n \equiv 0 \mod V$. The only thing we have to demonstrate for operations to be identical is that for $e_k^{(j)}$ and e_i there exists a bijective mapping between (j, k) and i.

Finally, if every vector in the vectorisation contains "phantom" elements starting from the $k - th$ to $V - 1$ position, then the operation has to be adjusted to:

$$\begin{aligned} &\vec{r} = [w_1^{(1)}, \ldots, w_k^{(1)}, x, \ldots, x] i \\ &\quad \vec{\times} \cdots \vec{\times} [w_1^{(n/V)}, \ldots, w_k^{(n/V)}, x, \ldots, x] \\ &r = \vec{r}[0] \times \cdots \times \vec{r}[k] \end{aligned}$$

Proof. Let us consider a combination of all valid types for $\mathcal{T}(e)$, $\mathcal{T}(i)$ and $\mathcal{T}(f)$.

$\mathcal{T}(e) :: 0 \wedge \mathcal{T}(i) :: 0 \wedge \mathcal{T}(f) :: (0) \to 0$ non-vectorised case;

$\mathcal{T}(e) :: \tau \wedge \mathcal{T}(i) :: 0 \wedge \mathcal{T}(f) :: (\tau) \to \tau, \tau \in \mathbb{Z}_+ \cup \{\triangle\}$ In this cases, as f_{bin} is semantically correct and $f_{bin} :: (\tau, \tau) \to \tau$, this only difference that the previous case is non-scalar shape of f application. Using the same argument for assuming that the array components of e_1 and e are of shape s_{e_1} we reduce the proof to the previous case.

$\mathcal{T}(e) :: 0 \wedge \mathcal{T}(i) :: idx(k) \wedge \mathcal{T}(f) :: (idx(k)) \to \triangle$ As we know, we can permute the elements that we are reducing with f_{bin}. We need to show that the mapping between e_1 and $\mathcal{T}(e_1)$ elements is bijective for all e_1; that the number of scalar reductions is the same in transformed and original cases and that "phantom" elements do not affect the result.

First of all, the mapping between e_1 and $\mathcal{T}(e_1)$ is I_k, which is bijective by definition, and as e_1 is evaluated per every element in the index set, all the original e_1 are mapped. Secondly, "phantom" elements do not affect the result as every vector in $\vec{r'}$ contains exactly $u_k \texttt{ mod } V$ elements starting from the first position. As f_{bin} is correct, the operation is happening component wise, so the resulting vector is not affected and contains $u_k \texttt{ mod } V$ "real" values. Finally, the construction of r_2 guarantees that only "real" values are reduced.

Now, let us count the number of elements in a non-vectorised case, which would be $N = \prod_{j=0}^{\mathcal{D}(u)} u[j]$. The number of f_{bin} applications is $N-1$. Now, consider the $\vec{r}$ part of the transformation. Assuming that $u_k = p_k V + q_k$, where $0 < q_k < V$, the number of scalar elements reduced in $\vec{r}$ is $\frac{N}{u_k} p_k$. Each vector operation is the same as V scalar operations, so we have $\left(\frac{N}{u_k} p_k - 1\right) V$ scalar operations for this part.

For the $\vec{r'}$ part we have $\frac{N}{u_k} \cdot 1$ vector elements, where only q_k of them are non-phantom. It means that totally we have $(\frac{N}{u_k} - 1) q_k$ operations.

Finally, r_1 adds $V - 1$ operations to reduce $\vec{r}$, r_2 adds $q_k - 1$ operations to reduce $\vec{r'}$, and final call of f_{bin} adds one more operation. If we sum-up all together we will get:

$$\left(\frac{N}{u_k} p_k - 1\right) V + \left(\frac{N}{u_k} - 1\right) q_k + V - 1 + q_k - 1 + 1$$

which adds up to $N - 1$.

□

Another thing to notice regarding reduce is the fact that $\vec{r}$ might result in an empty range if $u_k < V$ in which case the neutral element is being returned. Also, the $\vec{r'}$ might result in empty range, if $u_k \texttt{ mod } V = 0$ in which case, again, the neutral element would be used. One of $\vec{r}$ or $\vec{r'}$ is not empty as $u_k \geq 1$.

Finally we are going to consider IF, APP, LET, FUNDEF and LETREC constructions. The correctness of IF follows directly from the correctness of `mask` operation. Correctness of APP follows directly from the definition of $\mathcal{C}$ under assumption that the function itself and its arguments are correct. The same reasoning holds for LET.

For FUNDEF, as $\mathcal{T}(f(a_1, \ldots, a_n) = e) = f(a_1, \ldots, a_n) = \mathcal{T}(e)$, we can see that correctness trivially holds under the assumption that e is correct.

The same reasoning holds for LETREC. However, please note that, as we might have recursive functions in the body of function definitions, for which we need to do a fixed point iteration, but as the co-domain of $\mathcal{C}$ the fixed point becomes trivial, we assume that an expression is correct and continue the induction.

5. Application

Now we are going to consider a small exampl of matrix multiplication to demonstrate how the transformation is applied on the real

code. We are going to consider a single function, assuming that it is being called in some LETREC construct.

```
matmul(a,b) =
  map i < [N]
    map j < [N]
      reduce k < [N] (+) a[i++k] * b[k++j]
```

Both arguments a and b are of shape $[N, N]$. Now let us consider that the layout type inference is done, and we have the following types for the transformed expressions:

```
matmul(a, b):: (2,1) → 0 =
  map i :: 0 < [N]
    map j :: 0 < [N]
      reduce k :: idx(1) < [N]
             (+) a[(i ++ k) :: idx(2)]:: △
                 * b[(k ++ j) :: idx(1)]:: △
```

Now we can apply $\mathcal{T}$ on `matmul` which result in the following program:

```
matmul(a,b) =
  map i < [N]
    map j < [N]
      let
        rv1 = reduce
                k < [N/V] (vplus)
                vmul (vsel (i++k, a),
                      vsel (k++j, b));
        rv2 = reduce
              k < [if N mod V == 0 then 0 else 1]
              (vplus) vmul (vsel (i++[N/V], a),
                            vsel ([N/V]++j, b));
      in
        (reduce i < [V] (+) rv1[i])
        + (reduce i < [N mod V] (+) rv2[i])
```

Built-in vector operations are denoted as `vplus` and `vmul` for addition and multiplication accordingly. In the final step of the `reduce` transformation we usde the fact that both `rv1` and `rv2` reurn 1-d arrays of shape $[V]$. That allows us to replace slicing `[*,i]` with a standard selection `[i]`.

6. Related Work

The idea to modify data layouts by means of compiler transformations is not new. There was a number of works trying to improve a cache behaviour or streaming through GPUs. Some of the works attempt to solve a very similar problem of transforming data layouts for better SIMD usage. However, despite the fact that the goals are very similar, to our knowledge none of the existing works attempted to consider layout modifications as a whole program transformation and to demonstrate why it might be correct.

Tiling optimisations like [1] changes the order of the iterations in a given loop nest. Such an optimisation technique might be not sufficient for effective SIMD code generation. We have shown this in [12] at the example of N-Body – a solution with the best spatial locality grants less vectorisation opportunities than tiling across the first axis of a 2-d array, with subsequent reorganisation of the data structure.

A number of works use a polyhidra model to optimise data layouts for a given loop nest. On the one hand, polyhidra transformations is a very powerful tool that allows to make a number of things under the hood, in which case most of the considerations regarding correctness disappear due to the known correctness of polyhidra itself. On the other hand, it stays unclear how to apply such a transformation on the whole program, where different loop nests potentially grant different layouts. The information has to be propagated outside the loop-nests and the overall program has to be adjusted. Alternatively, layout changes are possible at the beginning of the loop nest and then restoring original layouts on exit, or even employing some of just-in-time compilation techniques. In that case it is not clear how to justify performance penalty for memory copying operations.

As an example of such works we consider [7] where layout transformations are used to reduce non-local accesses for localizable computations, aiming better performance on CMPs (chip multi-processors); or [4] where the overall goal is to optimise stencil computations for SIMD capable architectures.

Finally, we give a special attention to [10] where authors use data layout transformation to optimise grid applications for efficient execution on GPUs. The overall transformations they are studying are very similar to our work, however the application domain is bounded to one particular class of applications. The underlying programming language used in the study is C with custom extensions regarding propagation of the layout information. The last facts makes it hard to judge how hard would it be to generalise this technique on a wider class of applications.

7. Conclusions and future work

In this paper we have demonstrated how layout types can be used to transform data placement in memory alongside with the operations on these data structures. We have also demonstrated that the transformed program stays correct with respect to the scalar elements it computes. Finally we illustrated the proposed technique at the example of matrix multiplication.

These contributions constitute a significant stepping stone towards automating the layout transformation process. In particular our formulation of the correctness criterion plays a key role here. It enables formal reasoning about the correctness of our transformation. Besides creating the formal basis of this transformation it also is key for enabling a description of the layout transformation as a high-level program transformation. This, in turn, does not only benefit a description of the transformation in a high-level fashion, but it also can serve as a blue-print for an effective implementation.

Although we describe our technique in a functional setting, in principle, our technique can be applied in non-functional contexts as well. However, it would require quite some additional effort such as a program analysis to reveal data-parallel operations, several contextual restrictions on the legitimate statements and operations such as restrictions on pointer arithmetics, etc. Finally, the most challenging part would be to propagate the layout information across modules, not to mention potential semantical conflicts in languages where data descriptions and their layout in memory are tightly coupled.

When combining this work with our previous work on data layout inference (see [12]) a full automation of the entire process seems within reach. All that is missing is a cost model to estimate an expected performance of transformed programs. Even for simple examples the number of transformations can be quite big. For example, even for matrix multiplication two layout types are possible. And this number, at least potentially, grows with the dimensionality of the function arguments. We are confident that it will be possible to apply existing approaches towards cost models for functional languages to close this gap and, thus, to achieve a fully automated layout adjustment process.

References

[1] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the 2008 ACM SIGPLAN conference on Programming language design and implementation*, PLDI '08, pages 101–113, New York, NY, USA, 2008. ACM.

[2] Clemens Grelck and Sven-Bodo Scholz. SAC: A functional array language for efficient multithreaded execution. *International Journal of Parallel Programming*, 34(4):383–427, 2006.

[3] Clemens Grelck, Sven-Bodo Scholz, and Alex Shafarenko. A binding scope analysis for generic programs on arrays. In *Proceedings of the 17th international conference on Implementation and Application of Functional Languages*, IFL'05, pages 212–230, Berlin, Heidelberg, 2006. Springer-Verlag.

[4] Tom Henretty, Kevin Stock, Louis-Noël Pouchet, Franz Franchetti, J. Ramanujam, and P. Sadayappan. Data layout transformation for stencil computations on short-vector simd architectures. In *Proceedings of the 20th international conference on Compiler construction: part of the joint European conferences on theory and practice of software*, CC'11/ETAPS'11, pages 225–245, Berlin, Heidelberg, 2011. Springer-Verlag.

[5] Ralf Karrenberg and Sebastian Hack. Whole Function Vectorization. In *Code Generation and Optimization*, 2011.

[6] Chidamber Kulkarni, Francky Catthoor, and Hugo De Man. Advanced data layout optimization for multimedia applications. In *Proc. Workshop on Parallel and Distributed Computing in Image, Video and Multimedia Processing (PDIVM'00), IPDPS'2000*, pages 186–193, 2000.

[7] Qingda Lu, Christophe Alias, Uday Bondhugula, Thomas Henretty, Sriram Krishnamoorthy, J. Ramanujam, Atanas Rountev, P. Sadayappan, Yongjian Chen, Haibo Lin, and Tin fook Ngai. Data layout transformation for enhancing data locality on nuca chip multiprocessors. *Parallel Architectures and Compilation Techniques, International Conference on*, 0:348–357, 2009.

[8] Liu Peng, Richard Seymour, Ken ichi Nomura, Rajiv K. Kalia, Aiichiro Nakano, Priya Vashishta, Alexander Loddoch, Michael Netzband, William R. Volz, and Chap C. Wong. High-order stencil computations on multicore clusters. In *IPDPS*, pages 1–11. IEEE, 2009.

[9] Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, 1 edition, February 2002.

[10] I-Jui Sung, John A. Stratton, and Wen-Mei W. Hwu. Data layout transformation exploiting memory-level parallelism in structured grid many-core applications. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, PACT '10, pages 513–522, New York, NY, USA, 2010. ACM.

[11] Konrad Trifunovic, Dorit Nuzman, Albert Cohen, Ayal Zaks, and Ira Rosen. Polyhedral-model guided loop-nest auto-vectorization. In *Proceedings of the 2009 18th International Conference on Parallel Architectures and Compilation Techniques*, PACT '09, pages 327–337, Washington, DC, USA, 2009. IEEE Computer Society.

[12] Artjoms Šinkarovs and Sven-Bodo Scholz. Data layout inference for code vectorisation. HPCS'13. Accepted for publication in HPCS'13 in 2013, to appear. `http://ashinkarov.github.io/publications/data-layouts.pdf`.